

## 第二章 比特币

# 本章内容

---

## 2.1 比特币简介

## 2.2 基础知识

## 2.3 交易

## 2.4 脚本系统

## 2.5 公私密钥与地址

## 2.6 区块与链

## 2.7 共识

# 比特币简介

---

## ➤ 加密数字货币的诞生

- 2008年10月31日中本聪发表《比特币：一种点对点的电子货币系统》，随后在2009年1月中本聪实现了比特币系统的最初版本。
- 比特币的出现宣告了加密数字货币的诞生，为货币系统实现去中心化、隐私保护、提供可靠服务给出了一套完整的解决方案。

## ➤ 比特币引入了更具价值的区块链技术

- 区块链不仅可以用于开发一个分布式去中心化的加密货币系统，更让人们看到了其在商业应用场景中解决安全和信任问题的潜力。

# 本章内容

---

## 2.1 比特币简介

## 2.2 基础知识

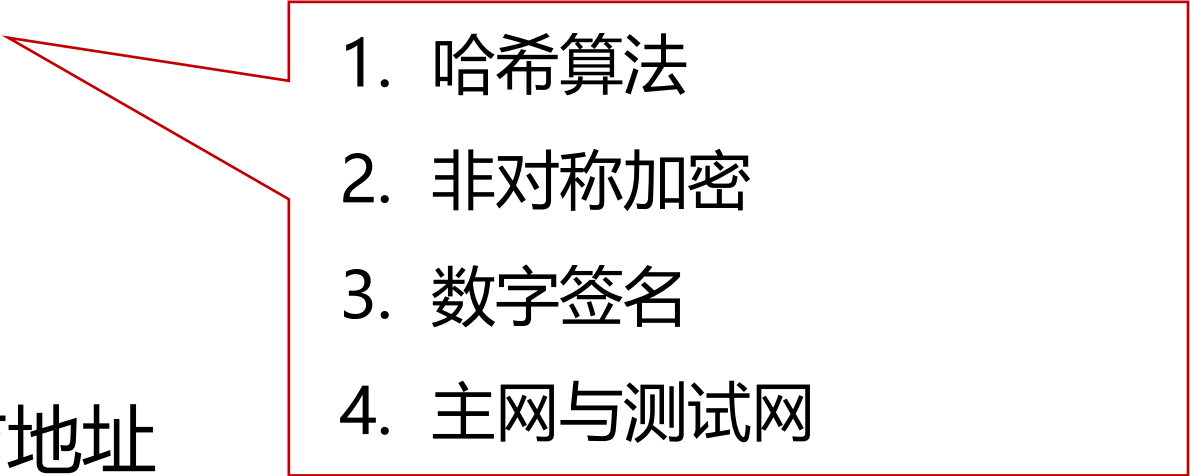
## 2.3 交易

## 2.4 脚本系统

## 2.5 公私密钥与地址

## 2.6 区块与链

## 2.7 共识

- 
1. 哈希算法
  2. 非对称加密
  3. 数字签名
  4. 主网与测试网

# 哈希算法

---

## ➤ 哈希 (Hash) 算法是什么

- 将任意长度的明文字符串映射成较短的二进制串，且不同的明文串映射后的值几乎没有冲突。
- 哈希值又被称为指纹或者摘要，因为它足够验证文件的完整性与正确性。
- 哈希函数本身是一种单射函数，所以哈希的整个过程并不可逆，如果需要对明文数据进行隐藏和保密，则需要使用加解密算法。

## ➤ 哈希碰撞

- 当不同输入的哈希产生相同的哈希值，则称为哈希碰撞。

hello,world -> 3CB95CFBE1035BCE8C448FCAF80FE7D9

hello world -> 5EB63BBBE01EEED093CB22BB8F5ACDC3

哈希算法例子

## ➤ 常见的Hash算法

- MD5, MD4由MIT的Ronald L. Rivest于1990年提出, 之后改进成MD5。
- SHA, SHA1由美国国家安全局 (NSA) 在1995年提出, 随后SHA2系列也被NSA公布出来。
- 目前, MD5与SHA1算法已经被破解。

## ➤ 区块链中Hash的应用

- 比特币采用了SHA2族的SHA2-256算法, 通常也简称为SHA256算法。  
比特币在挖矿和生成地址的时候都是采用了SHA256算法。
- 以太坊中采用了SHA3哈希函数族的来源——keccak族的keccak256 算法。

# 非对称加密

---

## ➤ 加密系统

- 加密系统一般包含三个部件：加解密算法、加密密钥、解密密钥。其中算法是公开的，而密钥则需要保护起来。加密的时候利用加密算法跟加密密钥，对明文进行加密，得到密文；解密的时候则利用解密算法和解密密钥，对密文进行解密，得到明文。

## ➤ 对称加密

- **加密密钥与解密密钥一样。**
- 该算法的优点是计算速度快，且空间占用小，效率高，加密强度也高，但是缺点是加解密双方要提前共享密钥，在这个共享的过程中存在泄漏密钥地风险，如何在不安全的环境下分享密钥也是一个研究的热点
- 常用算法：对称加密算法常用的是分组密码，即将明文切割为定长的数据块，并把每个数据块作为加密的基本单位，常见的该类算法有：DES、AES和IDEA等。

## ➤ 非对称加密

- **加密密钥与解密密钥不一样**，分别称为公钥和私钥，公钥是公开的，一般根据私钥产生，私钥则是由私人保护并持有，需要通过一定的算法来生成。
- 该算法的优点是无需提前共享密钥，在不安全的环境下也可以使用，缺点是速度慢，加密的强度不如对称算法大，虽然无需提前共享密钥，但也存在中间人攻击的可能。
- 常用算法：非对称加密算法一般由数学的经典难题来保障其安全，如椭圆曲线，以及大数的质因子分解等，常见的算法有RSA，SM2，椭圆曲线等。但是随着量子计算的到来，普遍认为RSA类算法将会被破解，椭圆曲线类算法更为推荐。

## ➤ 混合加密

- 混合加密结合了对称加密算法跟非对称加密算法的优点，可以实现更高效的加密通信。
- 常见做法：先用非对称加密协商确定一个临时的对称密钥，这一个临时的对称密钥是随机的，并且只有协商的双方才知道，在本次通信中使用这个一次性的对称密钥进行对称加密的通信。通信结束后，舍弃该对称密钥。

# 数字签名

---

## ➤ 数字签名是什么

- 数字签名是基于公钥加密技术实现的鉴别数字信息的完整性的算法，通过数学原理可以**保证被签名的数字信息的完整性和不可抵赖性**。在许多区块链系统中，交易的合法性通过数字签名技术来保证。

## ➤ 数字签名的使用方式

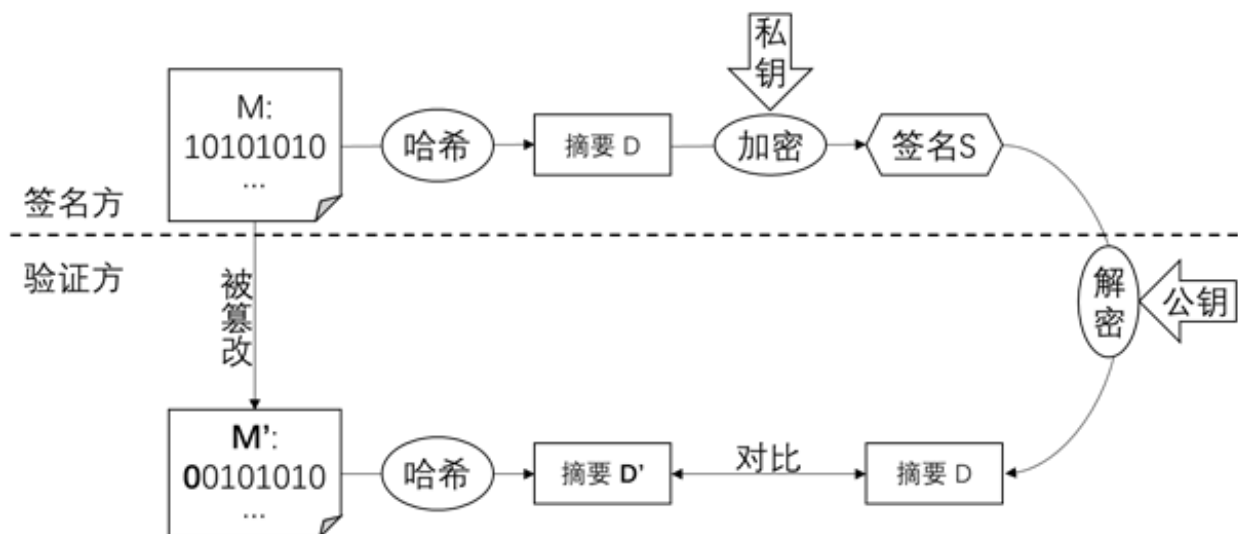
- 数字签名有两个重要步骤——签名和验证。其中，签名的密钥是不能够公开的，而验证的密钥只有必需的验证方可知。通常来说，验证方都是公开的，这也意味着数字签名的验证方法及验证的密钥是公开的。如果采用了对称加密的方法，这样的要求将导致密钥被公开给验证方，这是不可行的。而非对称加密可以很好地解决这个问题，签名方只需要通过私钥加密信息，验证方可以利用公钥进行解密。

## ➤ 签名和验证

- 如果直接对原有数据进行加密，这个过程的开销是十分巨大的。对此，通常采用了对原本信息的哈希值进行加密的方式来签名。对于一个消息  $M$  和其哈希摘要  $D$ ，签名者通过自身的私钥  $K_p$  生成对应的签名  $S = F(D, K_p)$ ，那么验证者便可以通过公钥  $K$  解密  $S$  得到接收到的消息  $M'$  的摘要  $D' = F'(S, K)$ ，通过消息  $M'$  产生的摘要  $D'$  与  $D$  的对比可以验证消息  $M$  的完整性和正确性。在哈希函数小节中我们知道，哈希的过程是具有抗篡改能力的，如果接收到的消息  $M'$  被第三方篡改过了，那么得到的哈希值很难再次对应（除非产生了碰撞）。同样，如果验证者已经接收到了签名  $S$ ，那么签名者也不能够再次修改数字信息，这体现了数字签名的不可抵赖性。

## ➤ 数字签名的例子

- 如图，对于检验方而言，通过检查数字签名S解密得到的摘要D和接收到的消息M'的摘要D'是否相同，便可以确定消息M的完整性和正确性。即使M被微小的修改为了M'，根据哈希函数的特点，得到的摘要D'也会完全不同。当然，签名的实际流程更为复杂一些，而且可能不包含加解密的过程，但都是通过接收到消息M的摘要D、公钥K，以及相对应的签名S来进行比对和验算。



简单的数字签名和检验流程

# 主网与测试网

---

## ➤ 主网

- 人为规定的比特币网络，在这个主网上流通的比特币才具有真正的经济价值，代表了比特币系统。

## ➤ 测试网

- 与主网相对的概念，一般用于开发和测试客户端、测试特定交易脚本等工作。在测试网中获取的比特币一般不具有实际的经济价值。往往由相对较多的节点和参与者，相对于本地的私链更能够模拟主网的环境。

## ➤ 本地测试网

- 称为Rogtest，通常只有本地节点，其余性质于测试网类似，可以用作本地测试。

# 本章内容

---

2.1 比特币简介

2.2 基础知识

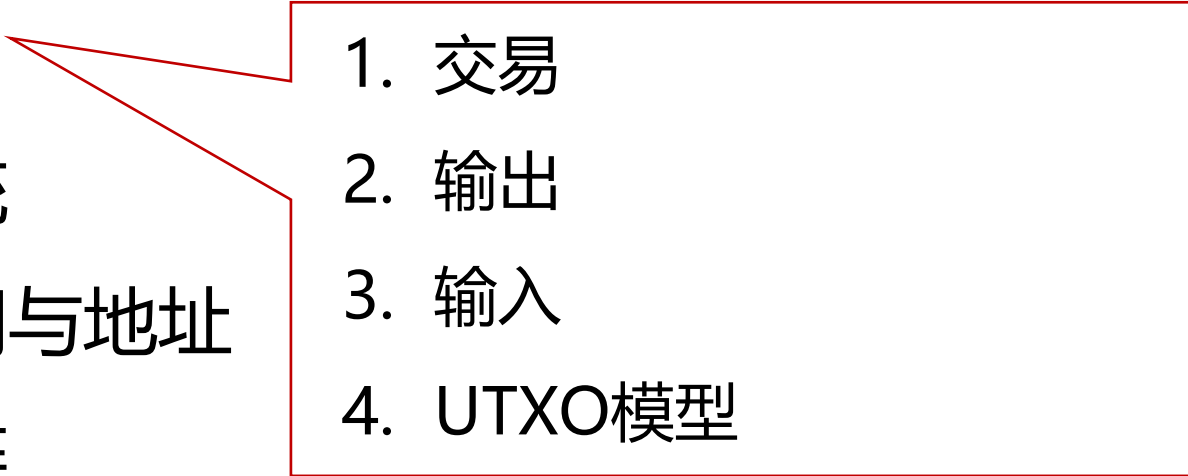
2.3 交易

2.4 脚本系统

2.5 公私密钥与地址

2.6 区块与链

2.7 共识

- 
1. 交易
  2. 输出
  3. 输入
  4. UTXO模型

# 账户与交易

---

## ➤ 账户

- 账户是区块链系统中数字货币的所有者，也称为地址。
- 账户之间的货币转移通过交易来实现，称为转账。

## ➤ 账户系统的经典模型

- **UTXO**，应用于比特币。
- **账户状态转移**，应用于以太坊。

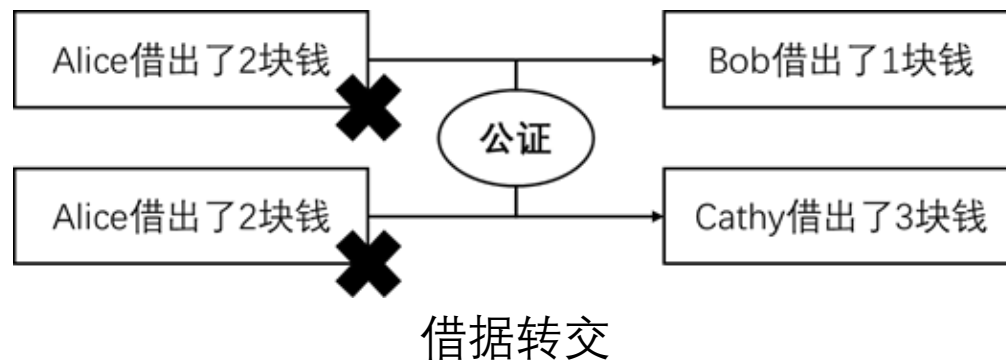
# 交易

## ➤ 账户余额

- 由UTXO表示（由之后的内容介绍）。

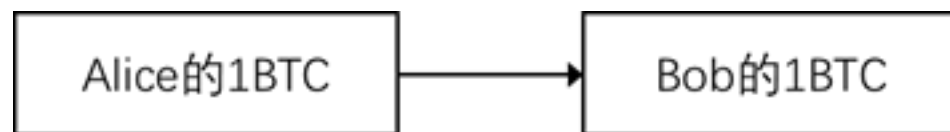
## ➤ 比特币的交易类似于借据交易

- 在这个过程中，借据上的借款数额虽然是可以继续划分的，但是借据本身是一个不可划分整体，而比特币的交易也是类似的，交易主要分为输入和输出两个部分。交易的输入表示了交易的比特币从哪里来，交易的输出则体现了交易中的这些比特币在交易结束后往哪里去。虽然交易中的比特币数值可以继续划分到很小的单位，但是交易的输入和输出本身都是单一的整体，不可划分。



## ➤ 比特币的交易（转账）

- 假设现在Alice要向Bob进行转账1个BTC，那么她必须使用一个交易来完成这个转账的动作。首先，Alice会将她所拥有的1个BTC放入到交易的输入中，再将这个交易的输出标记为Bob所有。那么，执行完这一交易之后，Alice就失去了放入输入的这一个BTC的所有权，而Bob得到了这个交易输出的BTC的所有权。在这个过程中，比特币系统中的所有参与者共同充当了公证人的角色，而密码学的原理充当了原本借据上具有法律效力的印章和签名。



Alice向Bob转账

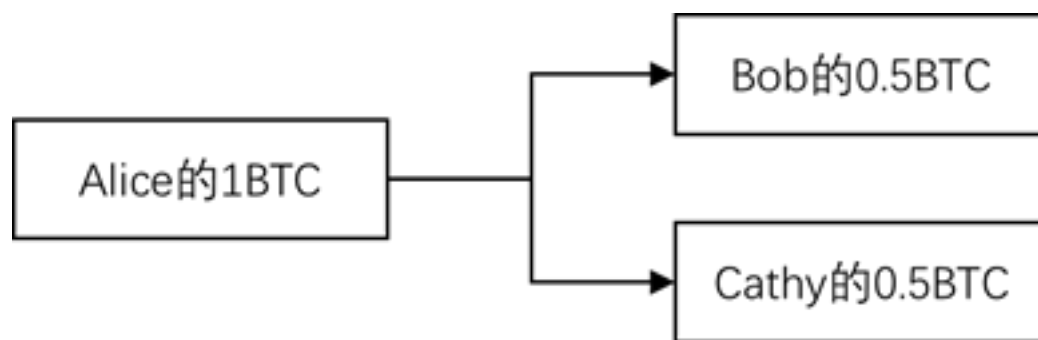
# 输出

## ➤ 交易的输出

- 在交易过程中，输出用来表示这笔交易的比特币去向。

## ➤ 交易可以有多个输出

- 在上面的Alice向Bob的转账中，交易的输出表明了交易中的1个BTC最终由Bob持有。在比特币的交易体系中，一个交易可以同时具备多个输出，如图，Alice可以利用一个交易同时向Bob和Cathy分别发送0.5个BTC。



多个输出的交易

## ➤ 输出的结构

- 为了证明转账后的结果，交易的一个输出包含了两个主要部分——输出的比特币数量和输出的比特币的所有者。如图，前者在比特币中通过一个64位的整数来记录，单位是比特币系统中的最小货币单位聪（Satoshi），大小为一亿分之一的比特币。后者我们使用了比特币的脚本系统来实现，我们这里可以简单理解输出的脚本就是对这个输出的比特币进行上锁，只有收款者Bob才有解开它的钥匙。



输出的结构

➤ 输出的结构

- 详细的输出结构格式可以参考下表，其中scriptPubKey的有关细节将会在脚本系统中介绍。

| 比特币交易输出结构    |          |       |                       |
|--------------|----------|-------|-----------------------|
| 数据项          | 类型       | 大小    | 描述                    |
| nValue       | 64位无符号整型 | 8字节   | 输出中的比特币数量，单位：聪        |
| scriptPubKey | 变长无符号整型  | 1-9字节 | 解锁输出脚本长度              |
|              | 字节串      | 不定    | 用于锁定的脚本，能够解锁脚本才使用这个输出 |

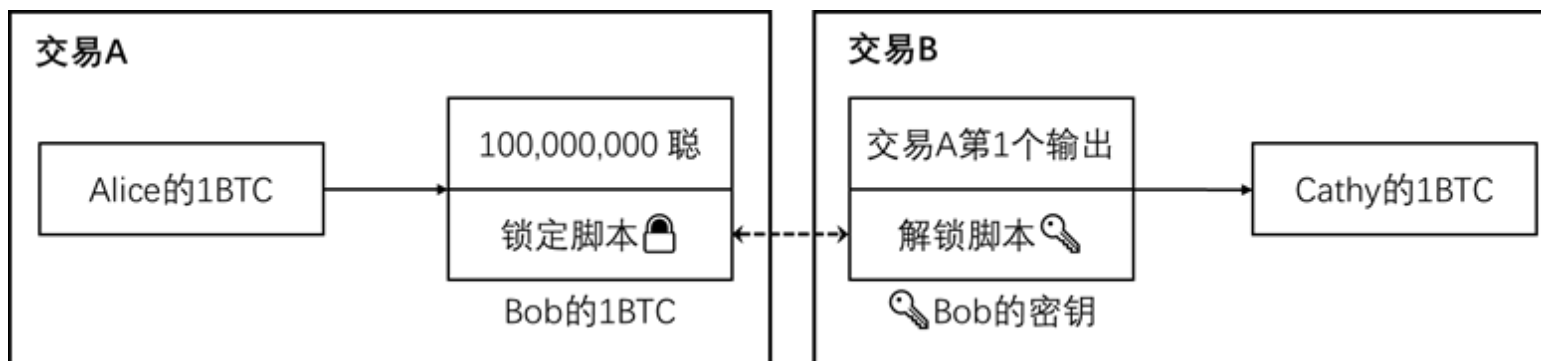
# 输入

## ➤ 交易的输入

- 交易输入的比特币不能凭空产生，必须引用和解锁一个已完成的其他交易的一个输出。
- 如图所示，在交易A中，Bob获得Alice转来的1个BTC；而在交易B中，Bob想把这1个BTC转给Cathy。所以在交易B中，他需要构造一个输入来引用交易B的这个输出。相当于他把交易A中的输出作为交易B中的输入。

## ➤ 交易的组成

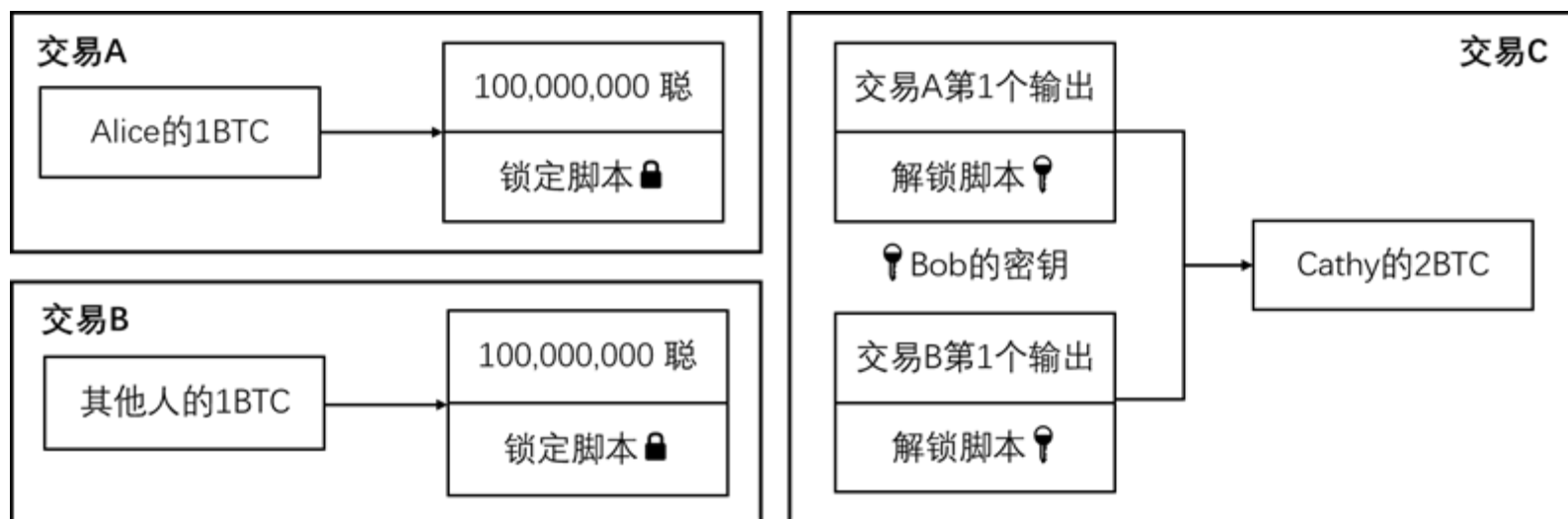
- 交易输入由两个部分组成——使用到的前一个交易输出的引用和用于解锁这个输出的一个解锁脚本。其中对输出的引用通过交易的哈希值和输出在这个交易中的位置决定，而解锁脚本通常使用了用户的密钥生成，只有用户本人才能够生成解锁脚本。



输入对前一个交易的输出的引用

## ➤ 多个输入的交易

- 同样地，交易的输入也可以有许多个。如图2.7所示，当Bob想要向Cathy转账2个BTC的时候，仅仅一个来自Alice转账得来的输出是不够的，所以还得加上另一个属于Bob的交易输出。



多个输入的交易

➤ 交易输入结构

| 比特币交易输入结构 |         |       |                 |             |
|-----------|---------|-------|-----------------|-------------|
| 数据项       | 类型      | 大小    | 描述              |             |
| prevout   | 哈希值     | 32字节  | 引用上一个交易输出       | 上一个交易的哈希    |
|           | 无符号整型   | 4字节   |                 | 输出在上一个交易的位置 |
| scriptSig | 变长无符号整型 | 1-9字节 | 解锁脚本长度          |             |
|           | 字节串     | 不定    | 用于解锁上一个输出的脚本    |             |
| nSequence | 无符号整型   | 4字节   | 序列号，在一些解锁的功能中使用 |             |

# UTXO模型

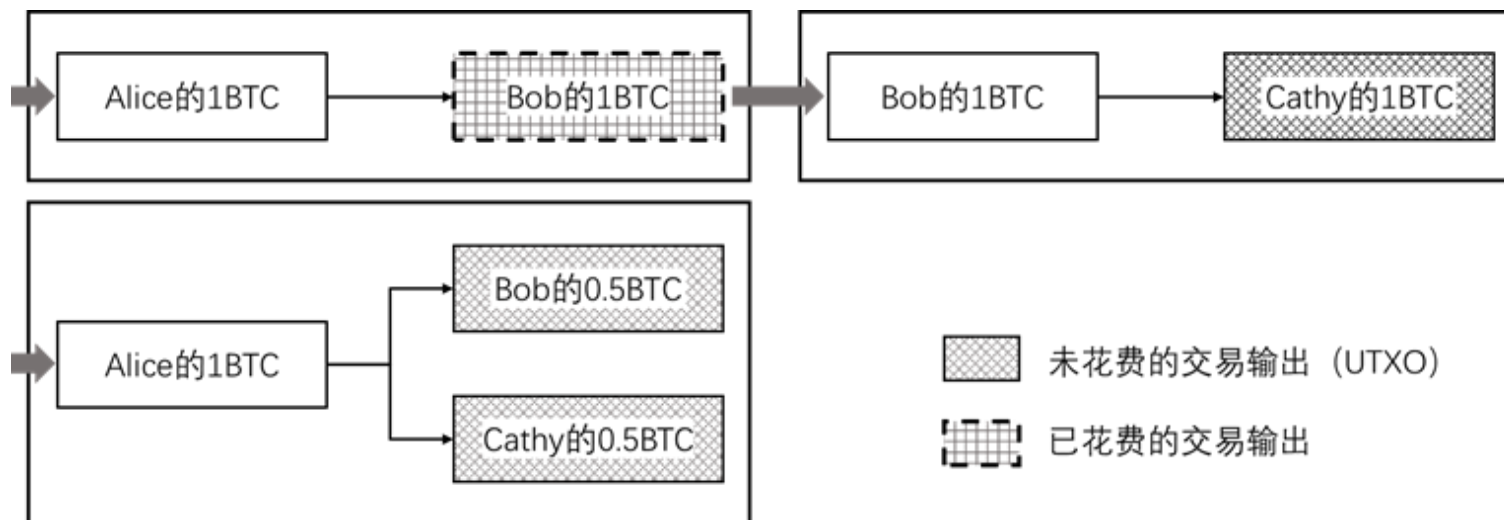
---

➤ 如何计算Bob拥有多少的比特币？

- 回到最开始Alice和Bob的交易，Bob通过自己密钥生成对应解锁脚本的方式得到这个交易输出的1个BTC的所有权，但Bob拥有了这个交易的输出的所有权不等于拥有这个比特币。
- 当Bob下次想要把这1个BTC转账给Cathy的时候，就可以通过类似的方式构建一个交易，把这个输出的1个BTC转账给Cathy，让Cathy获得新交易的输出的所有权。新交易完成后，原交易中Alice对Bob转账的输出依旧是存在的，所有权依然是Bob，然而Bob不可能再进行转账了——它已经被“花费”出去了。
- 因此，衡量Alice或者Bob到底拥有多少比特币，不能仅仅看她/他到底拥有了多少交易的输出，还要看这些交易输出到底有没有被所有者“花出去”。

## ➤ 利用UTXO模型计算Bob所拥有的比特币

- 如图所示，Bob的1个BTC的交易输出已经花费，而0.5个BTC的交易输出则尚未花费，代表了Bob的用户余额。在比特币的世界里，这些还没有花费出去的交易才能够真正地反映出一个用户所拥有的比特币，它们被称之为未花费交易输出（Unspent Transaction Output），简称为UTXO。
- 只要我们把比特币系统上的所有交易执行一遍，将所有由Bob才可以解锁的、同时也是还没有被“花费”的交易输出收集起来，我们就可以得到Bob用户的比特币余额



未花费与已花费的交易输出

## ➤ 可回溯的交易链条

- 在比特币的UTXO模型中，比特币并非凭空产生的，而是通过前人的交易得来的，前人的比特币又是通过更为前面的交易获取的。这样我们可以对任意的比特币不断回溯，直到最初产生比特币的起点（如何产生我们后面细讲）。在比特币的白皮书中，使用了“电子货币（指比特币）是一条数字签名链”的说法，讲的便是这样一个回溯的链条。在这个链条中，每一个节点都是一次比特币的交易，都是一次比特币所有者的签名认证与输出使用。当前任意一个比特币的持有证明，都是通过前面一层又一层交易的签名确认而得到的。最后，最新一笔交易的接收人可以通过验证输出的锁定脚本来确认他是这一比特币链条的所有者。

➤ 比特币的交易模型和结构

- 下表为交易数据结构细则，其中版本号可以实现在不修改原有交易的情况下实现对比特币交易的功能升级，而锁定时间则被用于定时锁定的比特币交易。这两个字段通常与比特币的一些复杂的脚本功能相关，这里不再继续展开。

| 比特币交易数据结构 |         |       |               |
|-----------|---------|-------|---------------|
| 数据项       | 类型      | 大小    | 说明            |
| nVersion  | 整数      | 4字节   | 交易的版本号        |
| vin       | 变长无符号整型 | 1-9字节 | 交易输入数量        |
|           | 数组      | 不定    | 交易的所有输入，按顺序排列 |
| vout      | 变长无符号整型 | 1-9字节 | 交易输出数量        |
|           | 数组      | 不定    | 交易的所有输出，按顺序排列 |
| nLockTime | 无符号整型   | 4字节   | 用于时间锁定脚本的时间戳  |

# 本章内容

---

2.1 比特币简介

2.2 基础知识

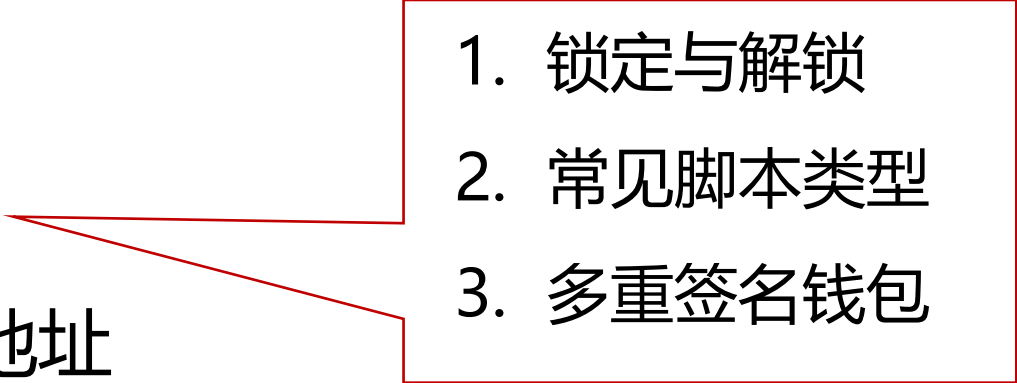
2.3 交易

2.4 脚本系统

2.5 公私密钥与地址

2.6 区块与链

2.7 共识

- 
1. 锁定与解锁
  2. 常见脚本类型
  3. 多重签名钱包

# 锁定与解锁

---

## ➤ 背景

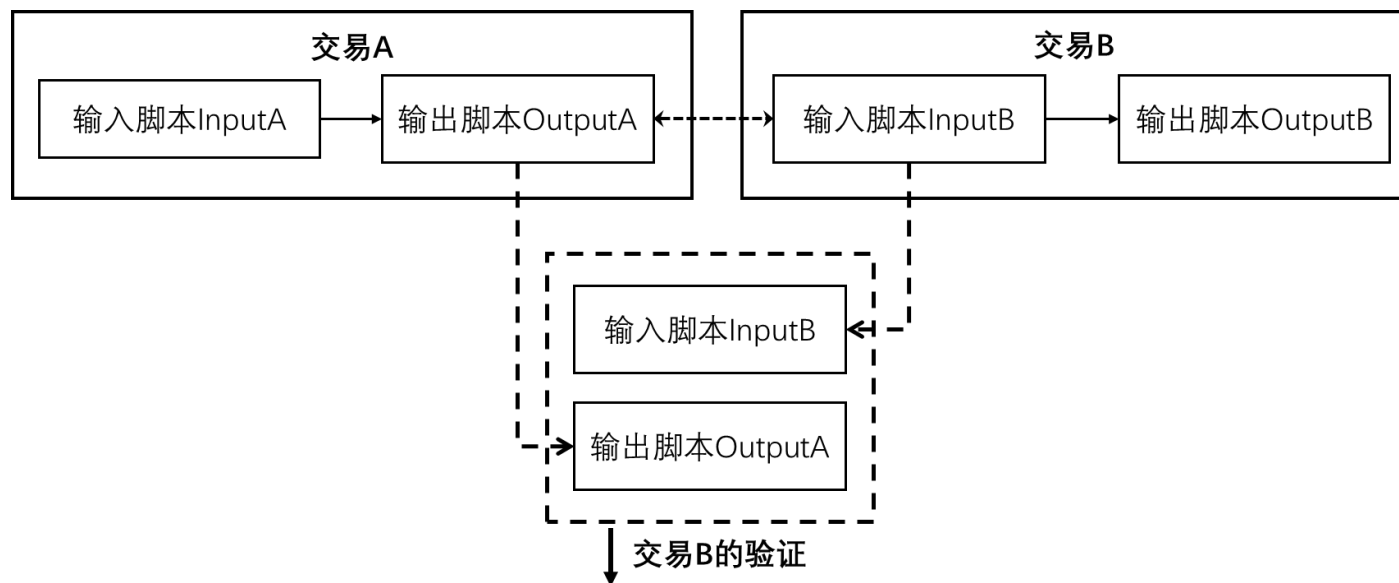
- 比特币**脚本**的执行是在一种基于栈的虚拟机中进行的。

## ➤ 用途

- 通过检查花费该UTXO 的交易的锁定脚本和解锁脚本，以**检查**这个**交易的合法性**。

## ➤ 实例

- 两笔前后依赖的比特币交易A、B，交易B的输入来自交易A的输出，即交易B花费的是A的UTXO。



# 常见脚本类型

---

## ➤ Pay-to-PublicKey (P2PK)

- 锁定脚本中包含了公钥，对应的解锁脚本需要给出公钥对应的私钥签名。

## ➤ Pay-to-PublicKey-Hash(P2PKH)

- 锁定脚本中包含了公钥的哈希，对应的解锁脚本需要另外给出对应哈希的公钥。
- 优点
  - 安全： P2PKH 向支付者交付的不是自己的公钥而是公钥的哈希值。
  - 节省手续费： 由于锁定脚本使用了长度更短的公钥哈希，可以一定程度地减少交易需要支付的手续费。

## ➤ Pay-to-Script-Hash (P2SH)

- 输出脚本中包含了一段脚本的哈希。
- 对应的解锁的输入脚本要给出一段序列化后的脚本。

## ➤ Null-Data

- OP\_RETURN 操作对应的脚本。

- 真实交易id:

ffa9be085f1d78acc0e0aa8303b3841fdb586b85e09af19b7e8d

665a10df0659对应的锁定脚本:

```
OP_RETURN  
OP_PUSHBYTES_39 6964.....1141
```

# 多重签名钱包

## ➤ 作用

- 个人签订一份合约，共同管理一笔钱。
- 只有得到至少M 个人的同意才能花费这笔钱。

## ➤ 实例

- 真实比特币区块链上的一个多方签名交易

7c0750818dc6c67761d591536a452e0091fe4262e7d51821c1f  
453df461e6ad4对应的解锁脚本：

```
OP_0  
OP_PUSHBYTES_71 3044.....c401  
OP_PUSHBYTES_71 3044.....a601  
OP_PUSHDATA1 5241.....53ae （序列化后的脚本，称为Redeem Script）
```

## ➤ 实例

- 真实比特币区块链上的一个多方签名交易

7c0750818dc6c67761d591536a452e0091fe4262e7d51821c1f  
453df461e6ad4对应的锁定脚本:

```
OP_HASH160  
OP_PUSHBYTES_20 08620a36625add363b20be66f8978f3f237f9c82  
OP_EQUAL
```

# 本章内容

---

2.1 比特币简介

2.2 基础知识

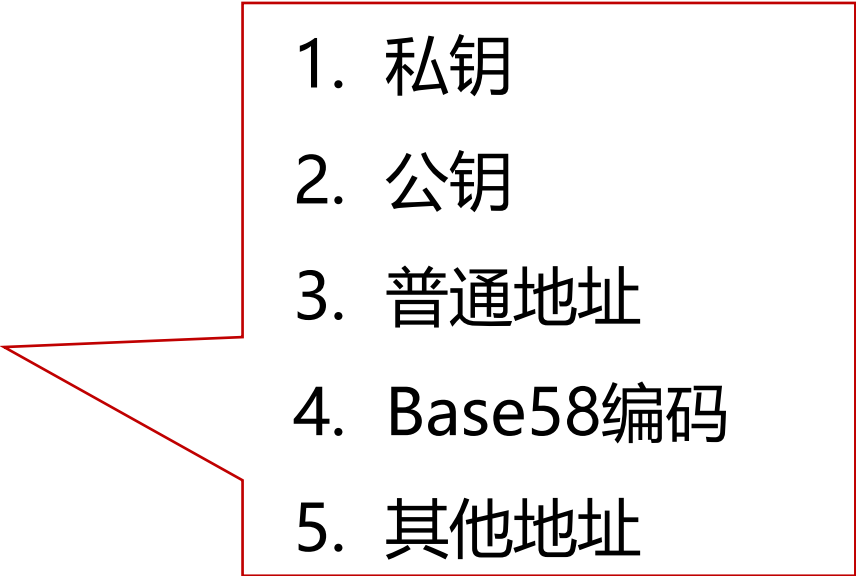
2.3 交易

2.4 脚本系统

2.5 公私密钥与地址

2.6 区块与链

2.7 共识

- 
1. 私钥
  2. 公钥
  3. 普通地址
  4. Base58编码
  5. 其他地址

# 公钥密钥与地址

---

## ➤ 背景

- 银行、网站
  - 用户的名称、密码一般是任意的、可自定义的。
  - 由权威的服务来保存用户与密码的认证关系
- 比特币等区块链系统
  - 用户的对外名称(地址)是根据用户的密码(私钥)生成的。
  - 在一个去中心化的系统中，利用非对称加密体系来建立密码与用户之间的映射关系。

# 私钥

---

## ➤ 比特币的加密

- 采用了椭圆曲线进行非对称加密。

## ➤ 私钥的定义

- 每个独立用户都拥有一个长度为32字节的私钥。
- 处于0x1 到0xFFFF FFFF FFFF FFFF FFFF FFFF FFFE BAAE DCE6 AF48 A03B BFD2 5E8C D036 4141 之间的任意64 位的十六进制整数。

# 公钥

## ➤ 定义

- 通过在一个特定的椭圆曲线上乘以椭圆曲线的一个固定点得到的一个坐标值。
- 比特币使用的这个特殊的椭圆曲线是由SECG (Standards for Efficient Cryptography Group) 提出来的secp256k1。
- 私钥通过计算可以得到一个长度为128 位的十六进制公钥，一般加上0x04 作为前缀区分。

## ➤ 例子

```
04 FB D0 A9 8C EF 18 0C DA A3 FC 2B 64 A4 CA 56 FB C8 E8 AF 5C C7
4E 82 0D A3 4E B9 DA B0 D4 6F F2 E3 85 87 07 92 9D 0A 9B EA CF 55
33 C2 2B F0 2D 34 E0 A6 EE 4F 1A 19 C1 CE AC 4F C4 25 18 10 A6
```

# 普通地址

---

## ➤ 定义

- 在公钥的基础上可以继续计算比特币的转账地址，这种地址一般被称之为普通地址。
- 对应的转账脚本则是P2PKH（Pay-to-PublicKey-Hash）。

## ➤ 实例

- 首先对公钥进行采用SHA256 算法进行哈希计算。

```
53E8E074EA414244B104A43DDA0D3DB7D7939EA767F2DE1D922DE9  
C26E96E5E9
```

- 对上述结果再进行一次RIPEMD160 的哈希计算。

```
0CC2D07F3801D43FD444EB842645FE30C3044B30
```

- 计算32 位校验和，对上面的哈希结果再进行两次SHA256。

```
82B6D0CB245031B7B199E32DDDFB45CD04812FC219F1549FE7F0D2  
DBB4414A99
```

## ➤ 实例

- 取前4 个字节（也就是前八个十六进制数）作为校验码。

```
82B6D0CB
```

- 将主网前缀00、第2 步的RIPEMD160 的结果和校验和前4 个字节拼接起来得到地址的十六进制表示

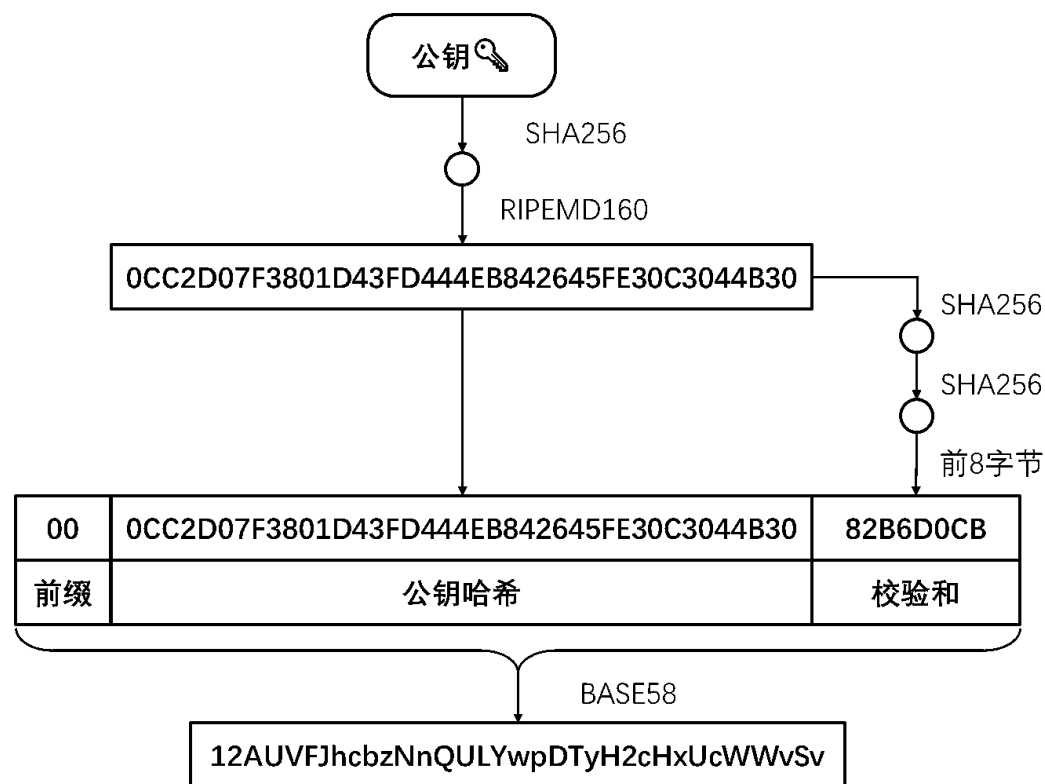
```
000CC2D07F3801D43FD444EB842645FE30C3044B3082B6D0CB
```

- 末尾的4 个字节提供了公钥哈希的校验能力。

# Base58编码

## ➤ 定义

- 十六进制可读性很差，为此比特币定义了Base 58 编码系统来对公私密钥进行编码。



# 其他地址

- 利用前缀来对各种地址进行区分，避免因为转账地址的混淆带来的错误交易

| 比特币地址类别及对应前缀 |            |               |
|--------------|------------|---------------|
| 十六进制前缀       | Base58格式前缀 | 类型            |
| 00           | 1          | 公钥哈希地址        |
| 05           | 3          | 脚本地址          |
| 80           | 5          | 私钥 (未压缩公钥)    |
| 80           | K或者L       | 私钥 (压缩公钥)     |
| 6F           | m或者n       | 测试网公钥哈希地址     |
| C4           | 2          | 测试网脚本哈希地址     |
| EF           | 9          | 测试网私钥 (未压缩公钥) |

# 本章内容

---

2.1 比特币简介

2.2 基础知识

2.3 交易

2.4 脚本系统

2.5 公私密钥与地址

2.6 区块与链

2.7 共识

- 
1. 区块
  2. 区块头
  3. Merkle树
  4. 区块链

# 区块与链

---

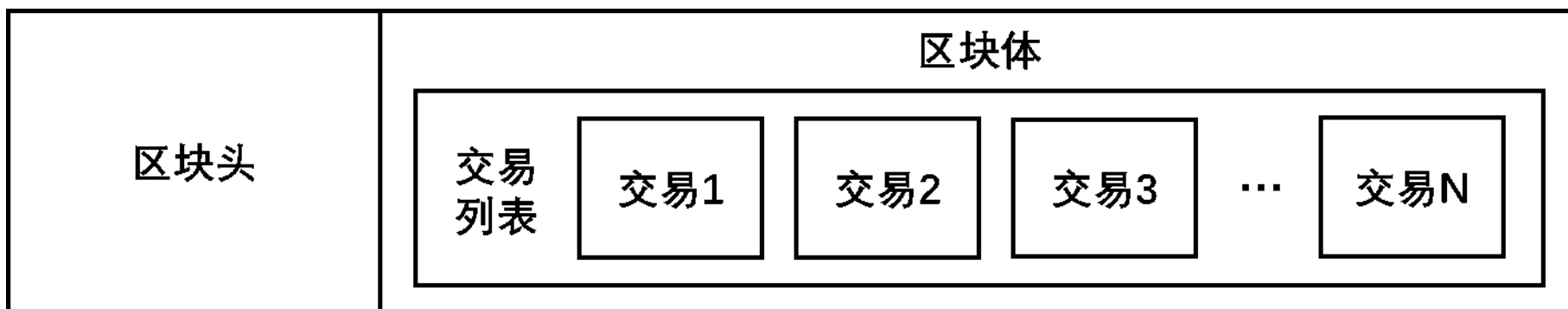
## ➤ 背景

- 交易需要得到验证和执行才能够正式被区块链系统确认。
- 如果每一个交易都要被单独确认的话，整个系统的开销将会非常巨大。
- 比特币系统中采取了批量确认交易的方法，形成区块。

# 区块

## ➤ 定义

- 区块链系统理解为一个数字账本，那么一个区块便是账本上的一页。
- 记录了相对独立的一批交易记录，并在对这些交易进行一次确认。
- 组成
  - 区块头 (Header)：包含了这个区块的所有信息
  - 区块体 (Body)：存放交易等详细数据



# 区块头

---

## ➤ 定义与作用

- 存放区块信息。
- 作为区块的摘要在网络间进行传输。

## ➤ 组成

- 区块版本。
- 前一个区块的哈希。
- 交易哈希：交易列表的哈希值。
- 产生的时间。
- 难度信息：区块产生的门槛难度。
- Nonce值：用于动态调整以产生满足门槛条件的合法区块。

## ➤ 比特币的区块头数据结构

| 表2.5 比特币的区块头数据结构 |        |              |
|------------------|--------|--------------|
| 源码中的命名           | 大小（字节） | 说明           |
| nVersion         | 4      | 区块遵守的版本      |
| hashPrevBlock    | 32     | 前一区块的哈希值     |
| hashMerkleRoot   | 32     | 区块中交易列表的哈希值  |
| nTime            | 4      | 产生区块的时间      |
| nBits            | 4      | 区块产生的难度目标    |
| nNonce           | 4      | 凑齐难度目标的一个随机数 |

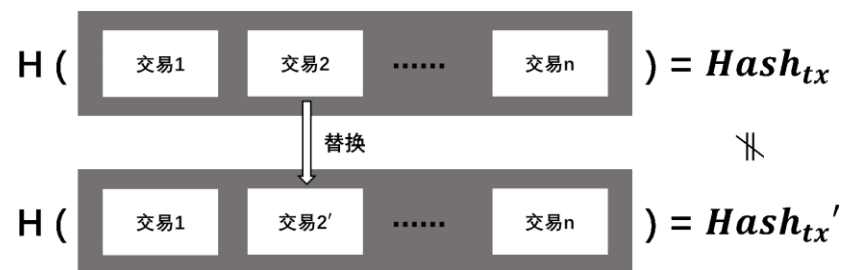
# Merkle树

## ➤ 背景

- 在区块头中，保存了区块中交易列表的哈希值。
- 当交易被篡改，则会得到完全不同的哈希值。

## ➤ 对整个交易列表进行哈希

- 不能够定位到具体出错的交易。
- 更换列表中的交易时，需要对所有的交易进行重新哈希。
- 必须掌握所有的交易才能够计算交易列表的哈希值。



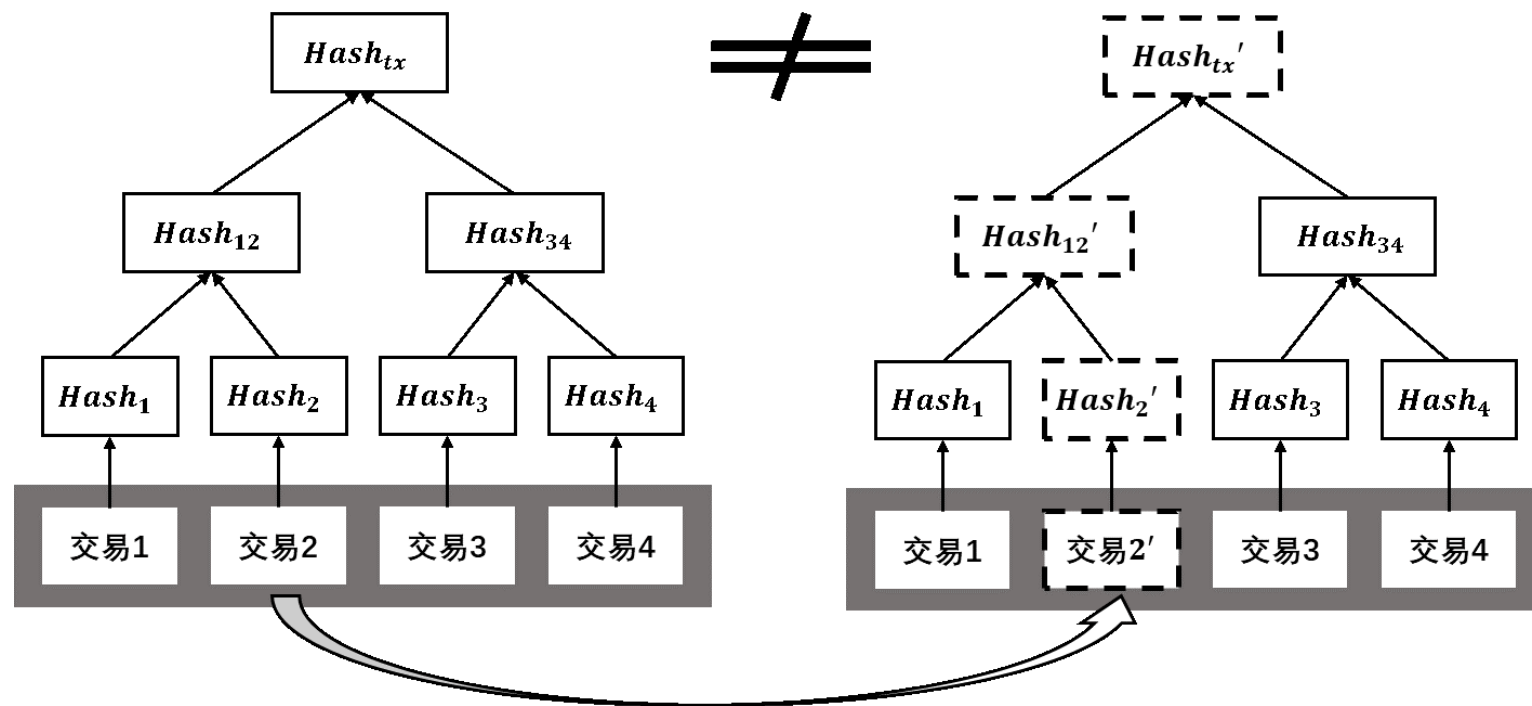
---

## ➤ 利用Merkle树计算哈希值

- 对列表中的每一个交易计算哈希值。
- 进行两两配对，计算两者的哈希值，形成上一层的哈希列表。
- 逐层计算直到得到最后的交易列表哈希值。

## ➤ 交易哈希值校验

- Merkle 树能够实现对交易列表的哈希值校验，确保列表中的元素不会被修改。



---

## ➤ 快速重新计算

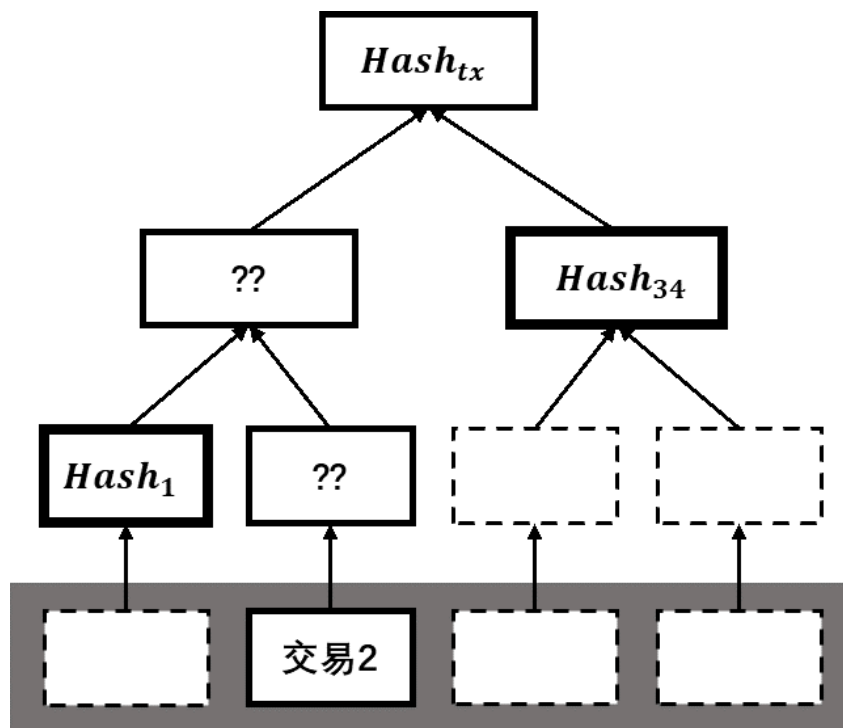
- 利用前一次计算得到的哈希值，只对被修改的树节点进行哈希重计算。
- 得到一个新的根哈希用来代表整棵树的状态。

## ➤ Merkle 树分支和轻节点验证

- 通过提供特定的叶子节点来完成对列表中任意一个元素的哈希证明。
- 从而避免对列表全部元素的访问。

## ➤ Merkle 树分支和轻节点验证

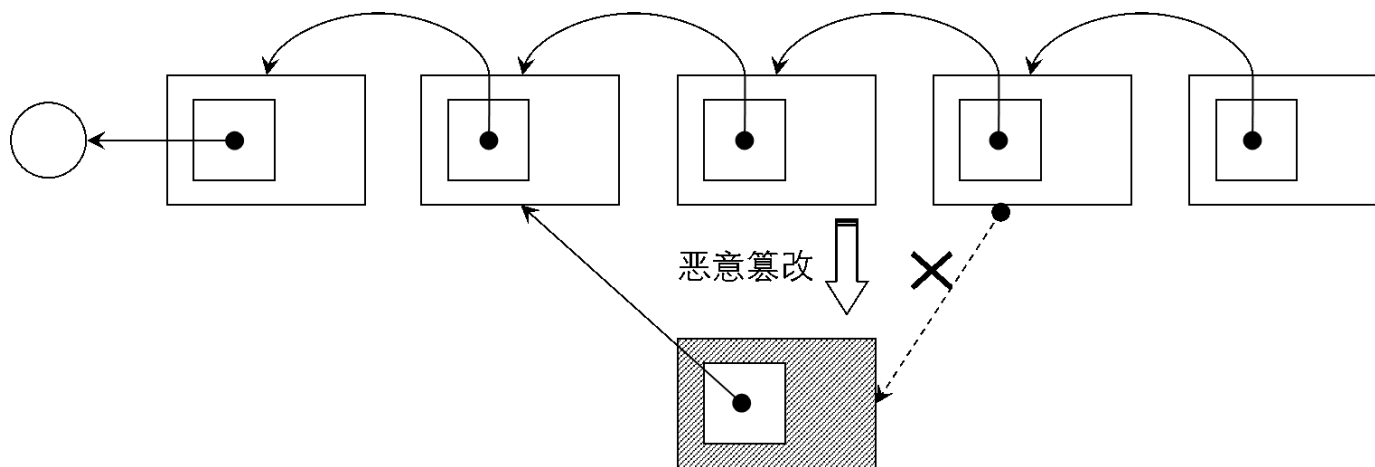
- 假设我们需要验证交易2 是否合法，只需要提供 $Hash_1$ 和 $Hash_{34}$ 即可完成。



# 区块链

## ➤ 背景

- 区块头中，包含一个特定的哈希值，用来指向上一个区块。
- 第一个区块被称为创世区块。
- 当前区块的高度
  - 创世区块开始到当前区块包含的区块数量
- 当区块链的中间一个区块受到恶意篡改时，就必须修改后面一个区块的内容。



# 本章内容

---

2.1 比特币简介

2.2 基础知识

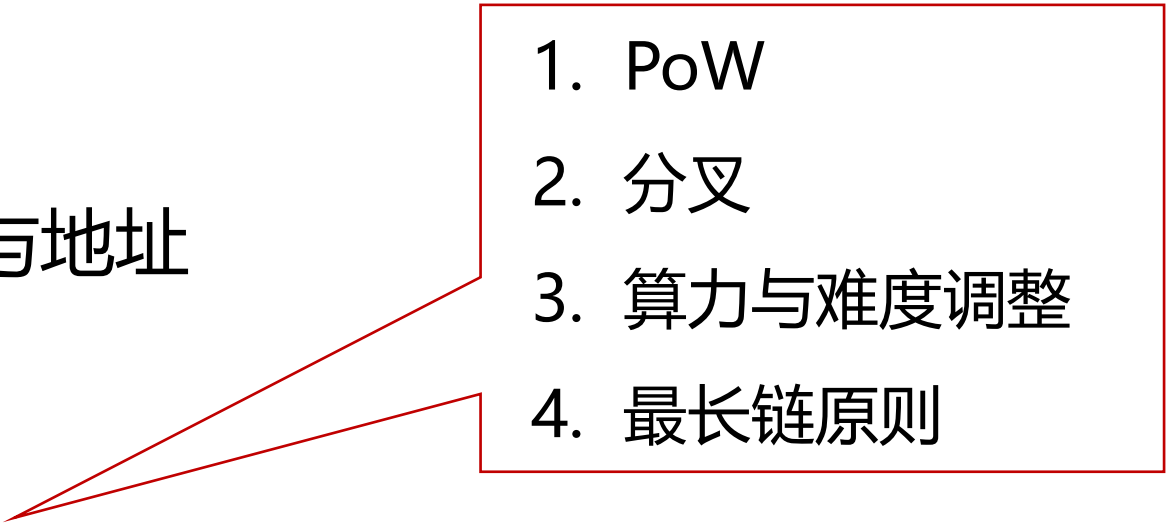
2.3 交易

2.4 脚本系统

2.5 公私密钥与地址

2.6 区块与链

2.7 共识

- 
1. PoW
  2. 分叉
  3. 算力与难度调整
  4. 最长链原则

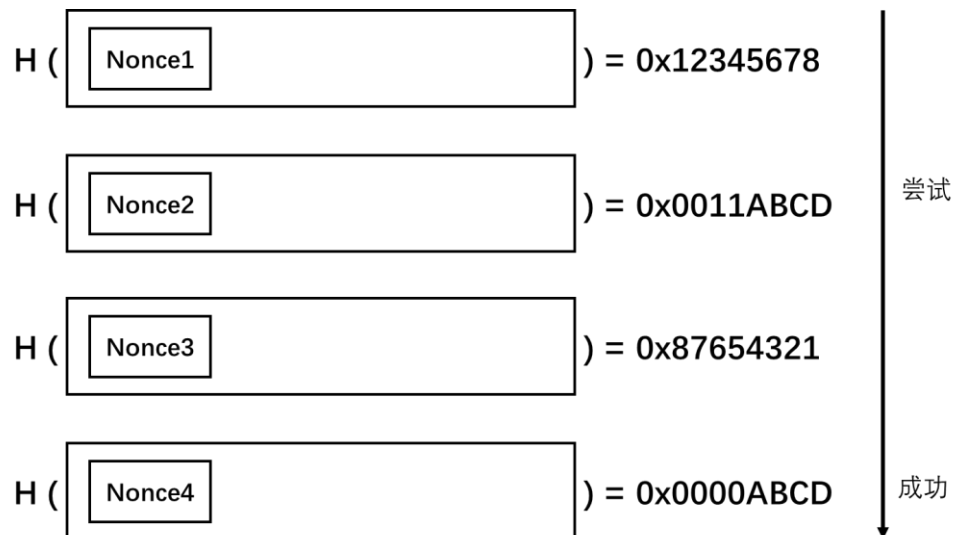
# PoW

## ➤ 背景

- 区块得到其他参与者的承认的过程便称为共识（Consensus）。

## ➤ 定义

- 对Nonce值进行动态调整，使得区块哈希值的前n位为0。
- 其中n被称作难度（Difficulty）。



---

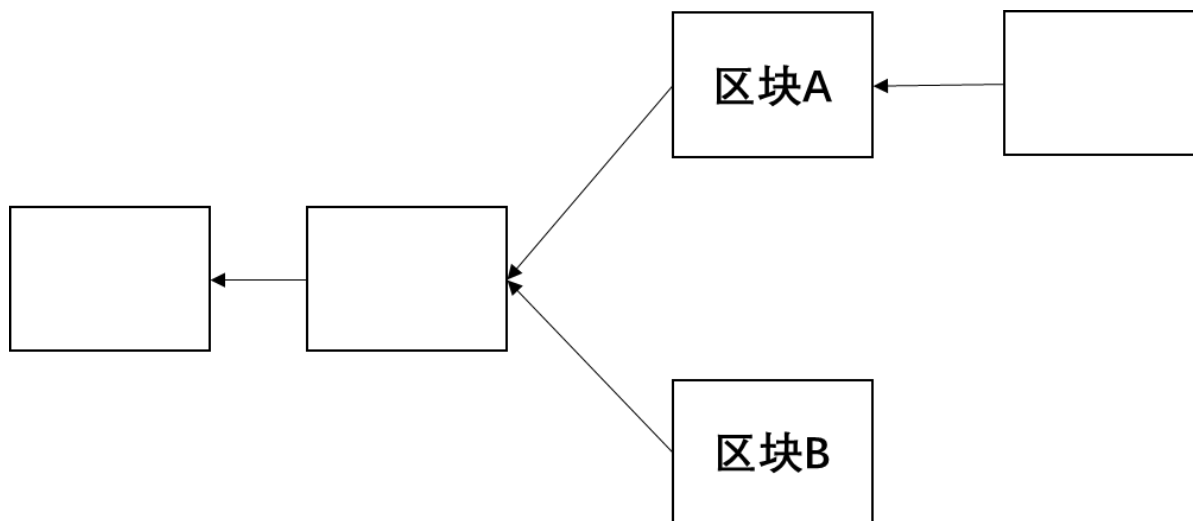
## ➤ 特点

- 哈希函数的单向性，这个Nonce 只能通过随机尝试的方式来找到。
- 比特币系统规定了打包区块可以得到一定数量的比特币作为奖励。
- 这些奖励是所有UTXO 中比特币的唯一来源。

# 分叉

## ➤ 定义

- 可能有两个不同的矿工在同一时间，或者相继很短的一段时间内生成两个同样高度的区块。



---

## ➤ 原因

- 挖矿的难度过低使得矿工出块的速度过快。
- 网络的通信时延过长，不知道新区块产生的矿工们任然在同一高度下继续进行挖矿。

## ➤ 存在的问题

- 分叉的两个区块中可能存在互相冲突的交易，导致UTXO被“双花”。
- Maximal Extractable Value

# 算力与难度调整

---

## ➤ 背景

- 哈希算力不断变化
  - 全网机器在固定时间内能够进行尝试的次数。
- 随着全网算力的提高，如果难度固定，那么产生新区块的时间间隔将会越来越短。

## ➤ 比特币难度调整

- 比特币网络会进行难度调整，将区块时间间隔稳定在10分钟。

# 最长链原则

---

## ➤ 背景

- 由于区块链分叉现象可能导致全网不一致。

## ➤ 比特币网络

- 使用了最长链原则来确定一个节点应当选择的分支。
- 最长链原则也保证了前序的区块不可能被随意篡改。

# 本章总结

---

- 了解比特币诞生背景
- 哈希算法与数字签名
- 非对称加密原理
- 比特币交易与账户模型
- 了解交易脚本、公钥与地址
- 区块数据结构
- pow共识